

WORKSPACE COMPILATION STUDIO: DEPLOYMENT ARCHITECTURE

Corporate Security Policy Assertion: This document defines the deployment mechanics for the multi-threaded orchestration environment. All referenced customer data layers have been tokenized to comply with Red Hat confidentiality compliance structures.

Section 1: Architectural Foundations

The Workspace Compilation Studio utilizes an asynchronous runtime matrix to interact with upstream document processing layers. By decoupling long-running network tasks from individual UI elements, the system can smoothly process parallel tracks without resource contention or race conditions.

The application is divided into two primary runtime patterns designed to optimize cloud processing efficiency:

- **fast.py (Fast Track Assembly Layer):** Designed for standard document sets, leveraging clean default endpoints to achieve maximum upload throughput.
- **deep.py (Deep Grounding Layer):** Designed for deep web search validation patterns, utilizing dynamic research wait holds and capping the final context window at the top 25 references.

Section 2: Pre-Processing Lifecycle & Automation

Before invoking either engine, files must be structured into an absolute workspace layout named according to account parameters (e.g., `Venella_2026/`). Raw office files must be passed through a headless LibreOffice server context to generate compliant PDFs, followed by uniform whitespace path cleanup using local terminal toolchains.

2.1 Normalization Invocations

Ensure files are executable and processed inside the targeted root before spin-up:

```
chmod +x convert.sh san.sh
./convert.sh /Users/jasona/account_plan/Venella_2026/
./san.sh /Users/jasona/account_plan/Venella_2026/
```

Section 3: Telemetry Dashboard & Run Protocols

The user tracking system outputs live progress statistics directly to a local, self-refreshing HTML visualizer matching PatternFly Dark Core style guides. The engine initializes this visual display page right after the pre-flight verification step clears, allowing administrators to monitor ingestion performance metrics across all worker pools in real time.

To execute the production-grade threads, verify path arrays inside `vars.py` and fire the corresponding daemon wrapper from a virtual environment console:

```
# Launch Fast Track Pipeline Processing Loop
python fast.py

# Launch Deep Research Network Scraping Loop
python deep.py
```